

TRABAJO PRÁCTICO INTEGRADOR N°1 — PARTE 1

Metodología de Sistemas II — TUP 2026

Centro Educativo "Educar para Transformar"

Equipo de trabajo: *Trangoni Fabián / Bruno Mazza*

Fecha: 11 de agosto de 2026

1. Especificación de Requerimientos del Sistema

1.1 Objetivos estratégicos

Código	Objetivo estratégico
OE-01	Iniciar operaciones en marzo de 2027 con procesos de gestión automatizados.
OE-02	Centralizar y garantizar la integridad y actualización de la información académica.
OE-03	Facilitar la comunicación y autogestión de padres, docentes y estudiantes.
OE-04	Controlar eficientemente los servicios extracurriculares (deportes, transporte, comedor).
OE-05	Asegurar el acceso a la información según rol y permisos (seguridad).
OE-06	Disponer de información oportuna para la toma de decisiones (reportes).

1.2 Requerimientos Funcionales (RF)

Redactados según sujeto + verbo + objeto + condición, de forma atómica y verificable.

Código	Requerimiento funcional	Módulo	Obj.
RF-01	Permitir a cualquier visitante consultar información institucional, niveles educativos y noticias sin autenticarse.	Web	OE-01
RF-02	Permitir a un usuario no logueado dejar una opinión sobre la institución.	Web	OE-03
RF-03	Permitir el registro y autenticación de padres y estudiantes para acceder a información restringida.	Web	OE-05
RF-04	Permitir completar y enviar un formulario de solicitud de inscripción desde la web.	Web	OE-01
RF-05	Permitir al administrador el ABM de alumnos (legajo, DNI, datos personales, nivel, curso, estado).	Gestión	OE-02
RF-06	Permitir inscribir a un alumno en hasta 2 deportes, validando que no existan choques de horario.	Gestión	OE-04

RF-07	Permitir inscribir a un alumno a uno de los 4 recorridos de transporte, sin duplicados.	Gestión	OE-04
RF-08	Permitir al administrador el ABM de profesores (legajo, DNI, especialidad, estado).	Gestión	OE-02
RF-09	Permitir asignar a un profesor las materias y cursos a su cargo.	Gestión	OE-02
RF-10	Permitir gestionar usuarios y asignar permisos según el rol (directivo, docente, padre, estudiante).	Gestión	OE-05
RF-11	Controlar y rechazar inscripciones duplicadas (deportes, transporte, comedor, materias).	Gestión	OE-02
RF-12	Permitir generar reportes por alumno, por docente y listados agrupados.	Gestión	OE-06

1.3 Requerimientos No Funcionales (RNF)

Código	Categoría	Requerimiento no funcional
RNF-01	Usabilidad	Interfaz web responsive, utilizable sin conocimientos técnicos.
RNF-02	Seguridad	Cifrado de credenciales y control de acceso basado en roles (RBAC).
RNF-03	Disponibilidad	El sistema debe estar disponible al menos el 99% del horario escolar.
RNF-04	Confiablez	Consistencia de la información, sin duplicados, transacciones atómicas.
RNF-05	Mantenibilidad	Código organizado en capas desacopladas por módulo.
RNF-06	Interoperabilidad	El backend expone servicios (API) reutilizables por web y app móvil.
RNF-07	Auditoría	Registro de quién y cuándo realiza altas, bajas y modificaciones críticas.

2. Clasificación del Sistema de Información

El sistema se clasifica como un Sistema Integrado de Información, compuesto por tres niveles funcionales:

- TPS (Transaction Processing System) — nivel operativo: procesa las transacciones diarias de los módulos Alumnos y Profesores (altas, inscripciones a deportes/transporte/comedor, asignación de materias).
- MIS (Management Information System) — nivel táctico/directivo: el módulo Administrador consolida los datos del TPS en reportes (por alumno, por docente, listados) para apoyar la toma de decisiones de la Dirección.
- Portal Web Institucional — canal público y privado por rol: información corporativa, formulario de inscripción, opiniones y acceso segmentado para padres, docentes y estudiantes.

No se clasifica como ERP, ya que no integra procesos transversales como contabilidad o RR.HH.; su alcance se limita al dominio académico, deportivo y de servicios estudiantiles.

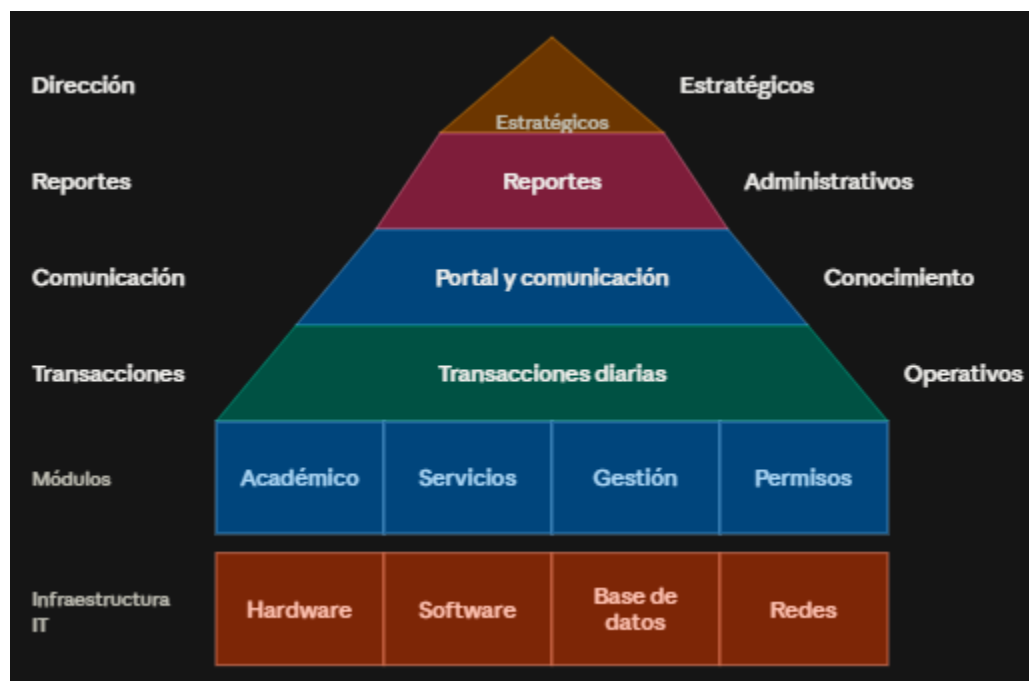
3. Arquitectura de la Información y de Software

3.1 Arquitectura de la Información

La arquitectura de la información se organiza como una pirámide de cuatro niveles, replicando el modelo clásico de sistemas organizacionales aplicado al centro educativo:

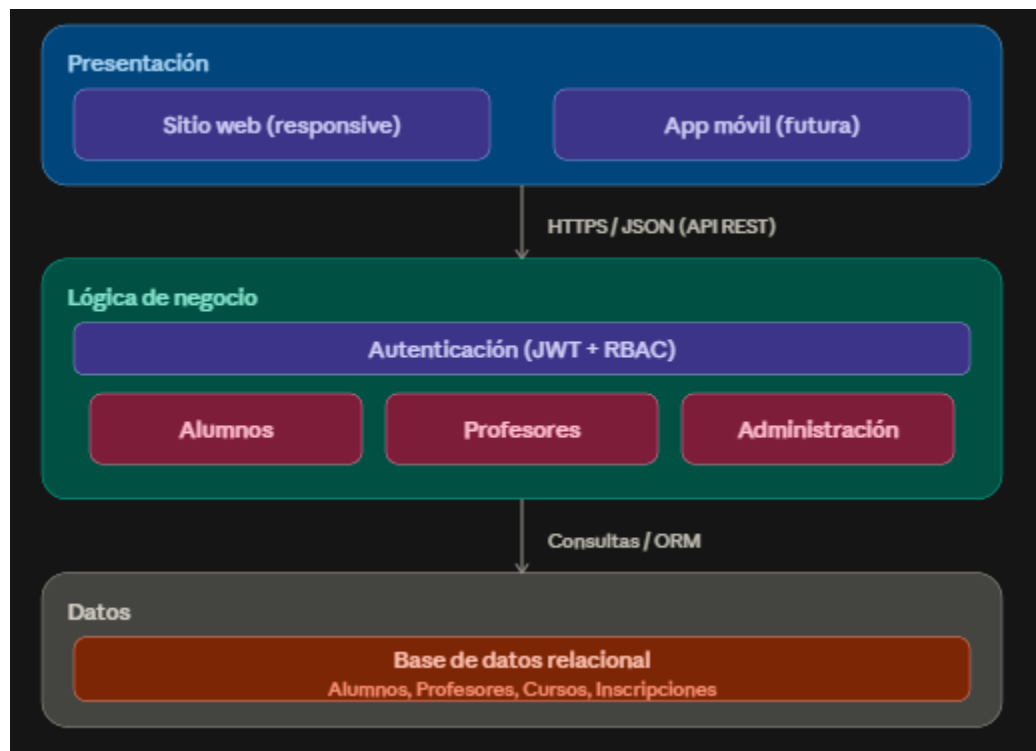
- **Nivel estratégico (Dirección):** consume reportes consolidados para decisiones sobre oferta académica y de servicios (RF-12).
- **Nivel administrativo (Reportes):** listados por curso, deporte, nivel y horario, generados por el módulo Administrador (RF-12).
- **Nivel de conocimiento (Comunicación):** portal web, opiniones y notificaciones que conectan a la institución con la comunidad (RF-02, RF-03).
- **Nivel operativo (Transacciones diarias):** altas de alumnos y profesores, inscripciones a deportes, transporte y comedor (RF-05 a RF-11).

Estos niveles se apoyan sobre los **módulos funcionales** (Académico, Servicios, Gestión, Permisos) y estos, a su vez, sobre la **infraestructura de TI** (hardware, software, base de datos centralizada y redes) que sostiene todo el sistema.



3.2 Arquitectura de Software

Arquitectura Cliente-Servidor multicapas (N-capas), con backend organizado bajo el patrón MVC / API REST, como fuente única de verdad para la Web y la futura App Móvil:



- Multicapas/MVC: separa presentación, lógica y datos (RNF-05); permite trabajo en paralelo por módulo.
- API REST única: Web y App consumen los mismos servicios, sin duplicar reglas de negocio (RNF-06).
- Monolito modular: adecuado para el alcance del TP, migrable a microservicios si el sistema escala.

4. Requerimientos funcionales

Integrante 1 Trangoni Fabián— Requerimientos: RF-02, RF-06, RF-07

RF-02 — Publicar opinión sin registro

A) Historia de Usuario

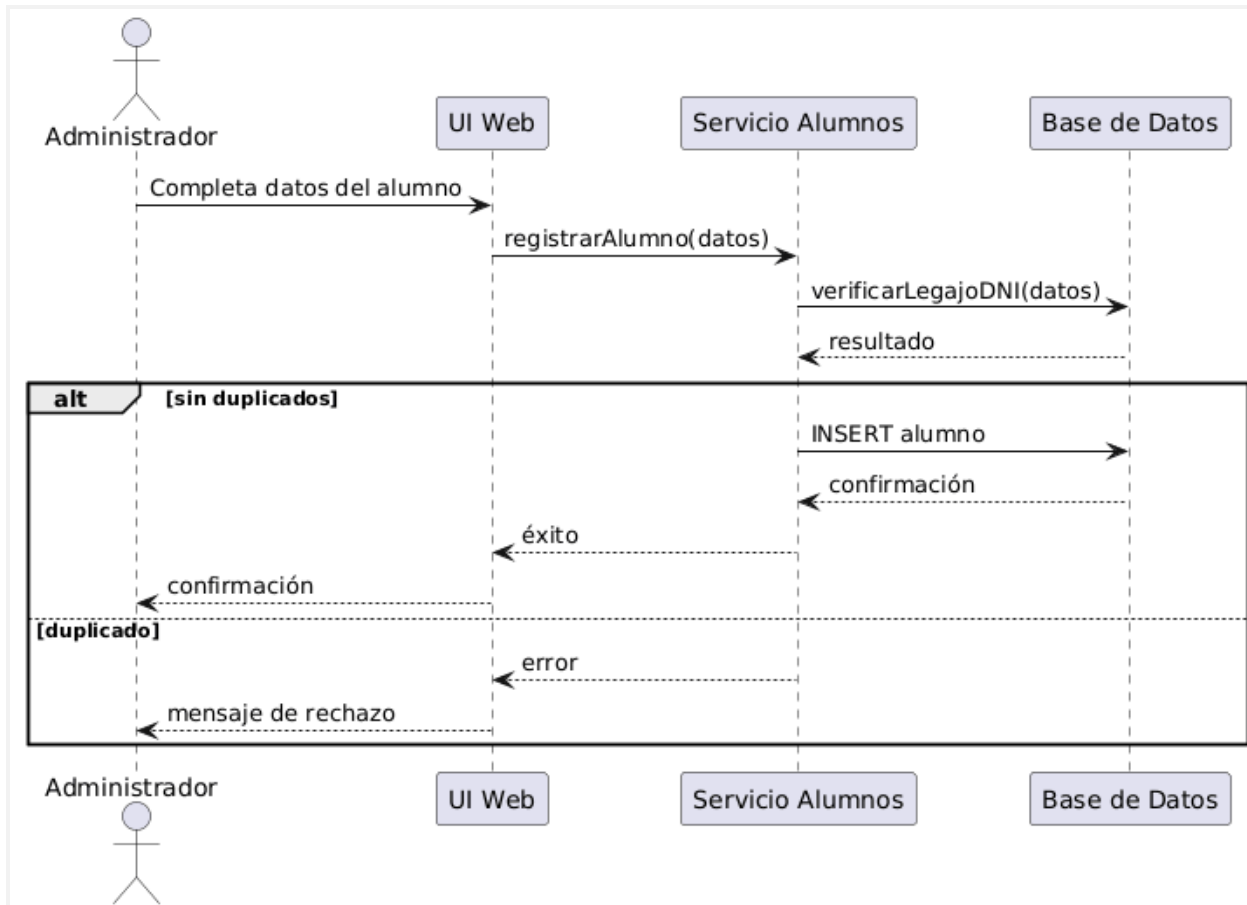
HISTORIA DE USUARIO — HU-01	
Número: HU-01	Usuario: Visitante web (no registrado)
Nombre historia: Dar de alta, modificar y consultar alumnos	
Prioridad en negocio: Media	Riesgo en desarrollo: Alto
Puntos estimados: 5	Iteración asignada: 1
Programador responsable: Fabián	
Descripción: Como administrador quiero dar de alta, modificar y consultar los datos de los alumnos, para mantener actualizada la información académica de la institución.	
Validación: El sistema exige legajo y DNI únicos; permite completar nombre, apellido, fecha de nacimiento, domicilio, teléfono, correo, nivel, curso y estado; al guardar, el alumno queda visible en el listado	

B) Caso de Uso

CASO DE USO — Publicar Opinión	
Autor: Fabián Trangoni	Fecha: 11/08/2026
Prioridad: 3	Actores: Administrador
Descripción: Permite crear, modificar y consultar el registro de un alumno en el sistema.	
Precondiciones: El administrador está autenticado con permisos sobre el módulo Alumnos.	
Flujo normal: 1.- Accede a "Gestión de alumnos" y selecciona "Nuevo alumno". 2.- Completa legajo, DNI, datos personales, nivel y curso. 3.- El sistema valida que legajo y DNI no estén repetidos. 4.- Registra el alumno y confirma al administrador.	
Flujo alternativo: 3.A.- Si el legajo o DNI ya existen, rechaza el alta e informa la duplicidad.	
Poscondiciones: El alumno queda registrado y disponible para inscripciones y reportes.	

C) Diagrama de Secuencia (PlantUML)

--



RF-06 — Inscripción a deportes (máx. 2, sin choque de horario)

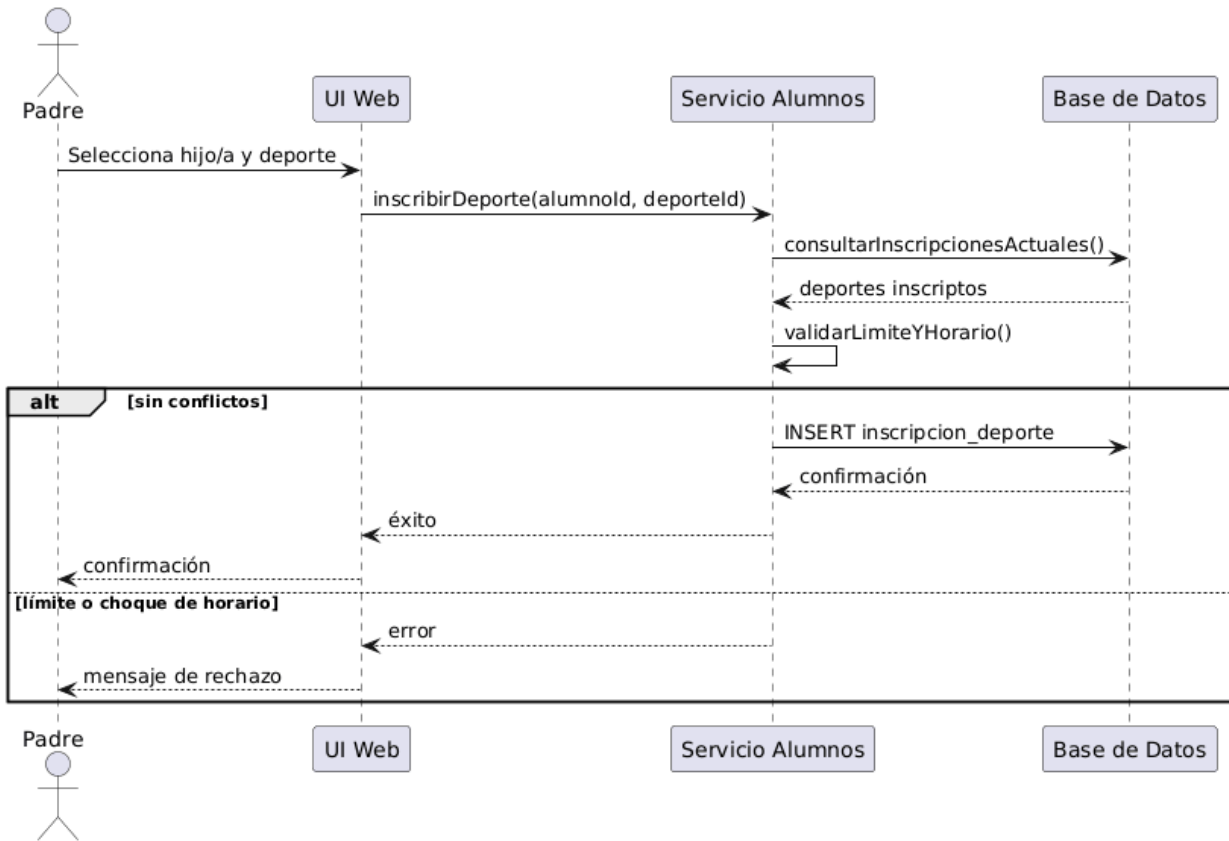
A) Historia de Usuario

HISTORIA DE USUARIO — HU-02	
Número: HU-02	Usuario: Padre/Madre
Nombre historia: Inscribir a mi hijo a actividades deportivas	
Prioridad en negocio: Alta	Riesgo en desarrollo: Medio
Puntos estimados: 8	Iteración asignada: 2
Programador responsable: Integrante 1	
Descripción: Como padre quiero inscribir a mi hijo en hasta dos deportes, para que participe sin que se generen conflictos de horario.	
Validación: No permite seleccionar un tercer deporte ni horarios superpuestos; al confirmar, queda registrada la inscripción.	

B) Caso de Uso

CASO DE USO — Inscribir Alumno a Deporte	
Autor: Trangoni Fabián	Fecha: 11/08/2026
Prioridad: 5	Actores: Padre/Madre
Descripción: Permite inscribir a un hijo/a en deportes respetando el límite de dos y la ausencia de choques de horario.	
Precondiciones: El padre está autenticado y tiene al menos un hijo/a registrado.	
Flujo normal: 1.- Selecciona al hijo/a y accede a "Inscripción a deportes". 2.- El sistema muestra deportes disponibles con horario y profesor. 3.- Selecciona un deporte. 4.- El sistema valida cantidad actual (< 2) y ausencia de choque de horario. 5.- Registra la inscripción y confirma al padre.	
Flujo alternativo: 4.A.- Si ya tiene 2 deportes, rechaza e informa el límite. 4.B.- Si hay choque de horario, rechaza e informa el conflicto.	
Poscondiciones: El alumno queda inscripto en el deporte, actualizándose su perfil deportivo.	

C) Diagrama de Secuencia (PlantUML)



```

@startuml
actor Padre
participant "UI Web" as UI
participant "Servicio Alumnos" as Svc
participant "Base de Datos" as DB

Padre -> UI : Selecciona hijo/a y deporte
UI -> Svc : inscribirDeporte(alumnold, deporteld)
Svc -> DB : consultarInscripcionesActuales()
DB --> Svc : deportes inscriptos
Svc -> Svc : validarLimiteYHorario()
alt sin conflictos
    Svc -> DB : INSERT inscripcion_deporte
    DB --> Svc : confirmación
    Svc --> UI : éxito
    UI --> Padre : confirmación
else límite o choque de horario
    Svc --> UI : error
    UI --> Padre : mensaje de rechazo
end
end
@enduml
  
```

RF-07 — Inscripción al servicio de transporte

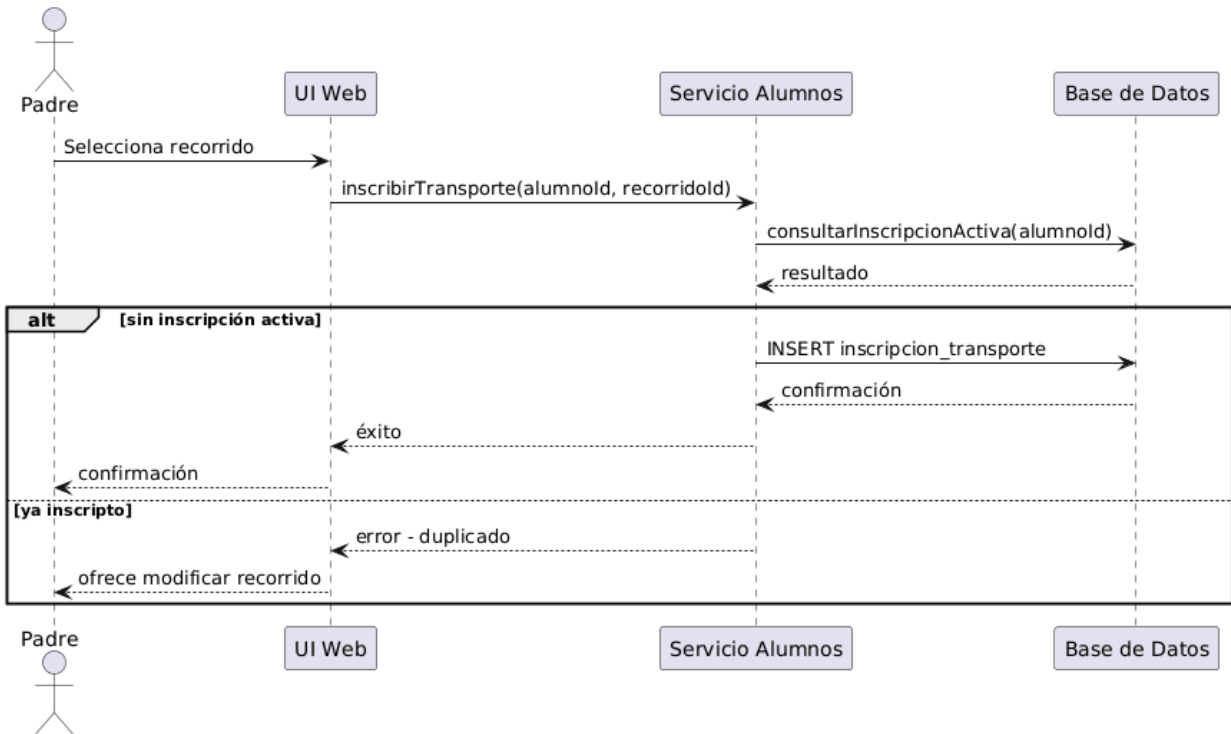
A) Historia de Usuario

HISTORIA DE USUARIO — HU-03	
Número: HU-03	Usuario: Padre/Madre
Nombre historia: Inscribir a mi hijo al servicio de transporte	
Prioridad en negocio: Alta	Riesgo en desarrollo: Bajo
Puntos estimados: 5	Iteración asignada: 2
Programador responsable: Integrante 1	
Descripción: Como padre quiero inscribir a mi hijo a uno de los recorridos de transporte, para asegurar su traslado diario.	
Validación: Permite elegir entre 4 recorridos, impide una segunda inscripción activa y confirma el recorrido asignado.	

B) Caso de Uso

CASO DE USO — Inscribir Alumno a Transporte	
Autor: Trangoni Fabián	Fecha: 11/08/2026
Prioridad: 4	Actores: Padre/Madre
Descripción: Permite asociar a un hijo/a a uno de los cuatro recorridos del servicio de micros.	
Precondiciones: El padre está autenticado y el alumno no posee inscripción activa a transporte.	
Flujo normal: 1.- Accede a "Servicio de transporte". 2.- El sistema muestra los 4 recorridos disponibles. 3.- Selecciona el recorrido correspondiente al domicilio. 4.- El sistema valida que no exista inscripción duplicada. 5.- Registra la inscripción y confirma.	
Flujo alternativo: 4.A.- Si ya posee inscripción activa, informa la duplicidad y ofrece modificarla.	
Poscondiciones: El alumno queda asociado a un único recorrido de transporte.	

C) Diagrama de Secuencia (PlantUML)



```

@startuml
actor Padre
participant "UI Web" as UI
participant "Servicio Alumnos" as Svc
participant "Base de Datos" as DB

Padre -> UI : Selecciona recorrido
UI -> Svc : inscribirTransporte(alumnold, recorridoId)
Svc -> DB : consultarInscripcionActiva(alumnold)
DB --> Svc : resultado
alt sin inscripción activa
    Svc -> DB : INSERT inscripcion_transporte
    DB --> Svc : confirmación
    Svc --> UI : éxito
    UI --> Padre : confirmación
else ya inscripto
    Svc --> UI : error - duplicado
    UI --> Padre : ofrece modificar recorrido
end
@enduml
  
```

Integrante 2 Bruno Mazza — Requerimientos: RF-09, RF-10, RF-12

RF-09 — Asignación de materias y cursos a profesor

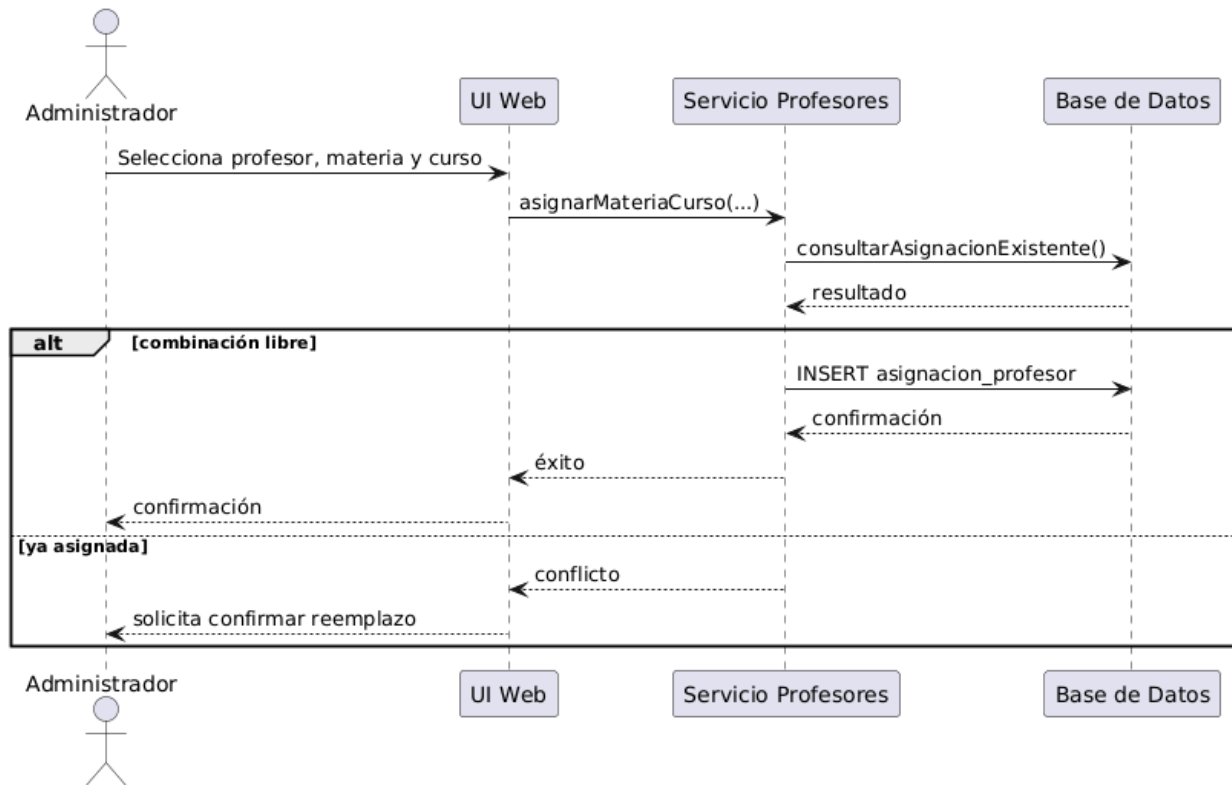
A) Historia de Usuario

HISTORIA DE USUARIO — HU-04	
Número: HU-04	Usuario: Administrador
Nombre historia: Asignar materias y cursos a un profesor	
Prioridad en negocio: Alta	Riesgo en desarrollo: Medio
Puntos estimados: 8	Iteración asignada: 2
Programador responsable: Integrante 2	
Descripción: Como administrador quiero asignar materias y cursos a cada profesor, para organizar la carga académica.	
Validación: Permite seleccionar profesor, materia y curso; contempla distintos profesores por curso para una misma materia.	

B) Caso de Uso

CASO DE USO — Asignar Materia/Curso a Profesor	
Autor: Bruno Mazza	Fecha: 11/08/2026
Prioridad: 5	Actores: Administrador
Descripción: Permite asociar a un profesor con las materias y cursos en los que dictará clases.	
Precondiciones: El profesor y la materia ya están registrados en el sistema.	
Flujo normal: 1.- Accede al perfil del profesor. 2.- Selecciona "Asignar materia/curso". 3.- Selecciona materia y curso. 4.- El sistema valida que la combinación no tenga profesor asignado. 5.- Registra la asignación y confirma.	
Flujo alternativo: 4.A.- Si ya existe asignación, informa el conflicto y solicita confirmar el reemplazo.	
Poscondiciones: El profesor queda vinculado a la materia y curso, disponible para reportes docentes.	

C) Diagrama de Secuencia (PlantUML)



```

@startuml
actor Administrador
participant "UI Web" as UI
participant "Servicio Profesores" as Svc
participant "Base de Datos" as DB

Administrador -> UI : Selecciona profesor, materia y curso
UI -> Svc : asignarMateriaCurso(...)
Svc -> DB : consultarAsignacionExistente()
DB --> Svc : resultado
alt combinación libre
    Svc -> DB : INSERT asignacion_profesor
    DB --> Svc : confirmación
    Svc --> UI : éxito
    UI --> Administrador : confirmación
else ya asignada
    Svc --> UI : conflicto
    UI --> Administrador : solicita confirmar reemplazo
end
end
@enduml
    
```

RF-10 — Gestión de usuarios y permisos por rol

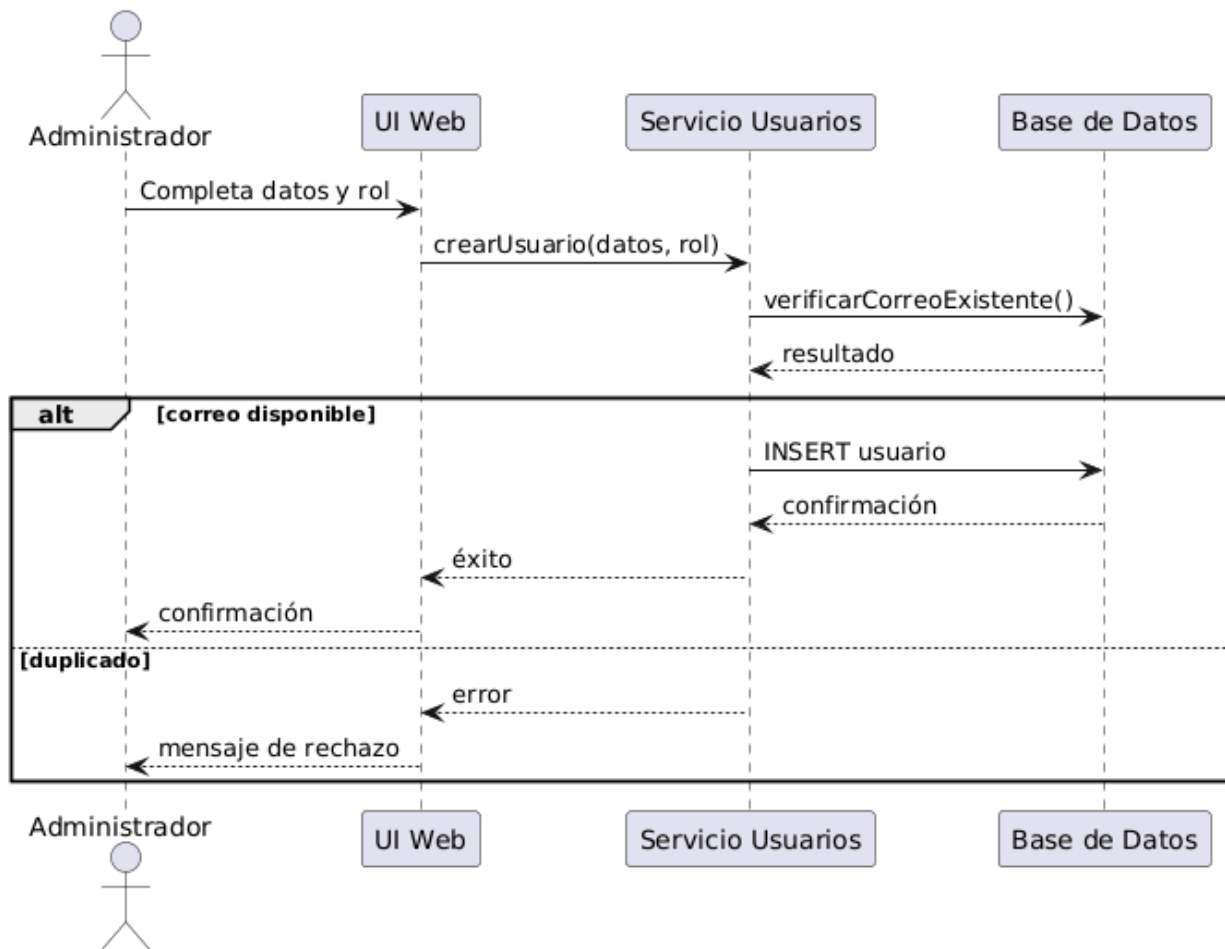
A) Historia de Usuario

HISTORIA DE USUARIO — HU-05	
Número: HU-05	Usuario: Administrador
Nombre historia: Gestionar usuarios y permisos del sistema	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alto
Puntos estimados: 13	Iteración asignada: 1
Programador responsable: Integrante 2	
Descripción: Como administrador quiero crear usuarios y asignarles un rol, para que cada persona acceda solo a su información.	
Validación: No permite crear un usuario sin rol; el usuario solo visualiza los módulos habilitados para su rol.	

B) Caso de Uso

CASO DE USO — Gestionar Usuarios y Permisos	
Autor: Bruno Mazza	Fecha: 11/08/2026
Prioridad: 5	Actores: Administrador
Descripción: Permite crear, modificar y desactivar usuarios, asignando el rol que determina sus permisos.	
Precondiciones: El administrador está autenticado con permisos de gestión de usuarios.	
Flujo normal: 1.- Accede a "Gestión de usuarios". 2.- Selecciona "Nuevo usuario". 3.- Completa datos y selecciona el rol. 4.- El sistema valida que el correo no esté registrado. 5.- Crea el usuario con su rol y confirma.	
Flujo alternativo: 4.A.- Si el correo ya existe, rechaza e informa la duplicidad.	
Poscondiciones: El usuario queda habilitado con acceso restringido a los módulos de su rol.	

C) Diagrama de Secuencia (PlantUML)



@startuml

actor Administrador

participant "UI Web" as UI

participant "Servicio Usuarios" as Svc

participant "Base de Datos" as DB

Administrador -> UI : Completa datos y rol

UI -> Svc : crearUsuario(datos, rol)

Svc -> DB : verificarCorreoExistente()

DB --> Svc : resultado

alt correo disponible

Svc -> DB : INSERT usuario

DB --> Svc : confirmación

Svc --> UI : éxito

UI --> Administrador : confirmación

else duplicado

Svc --> UI : error

UI --> Administrador : mensaje de rechazo
end
@enduml

RF-12 — Reporte de alumnos por deporte, nivel, horario y profesor

A) Historia de Usuario

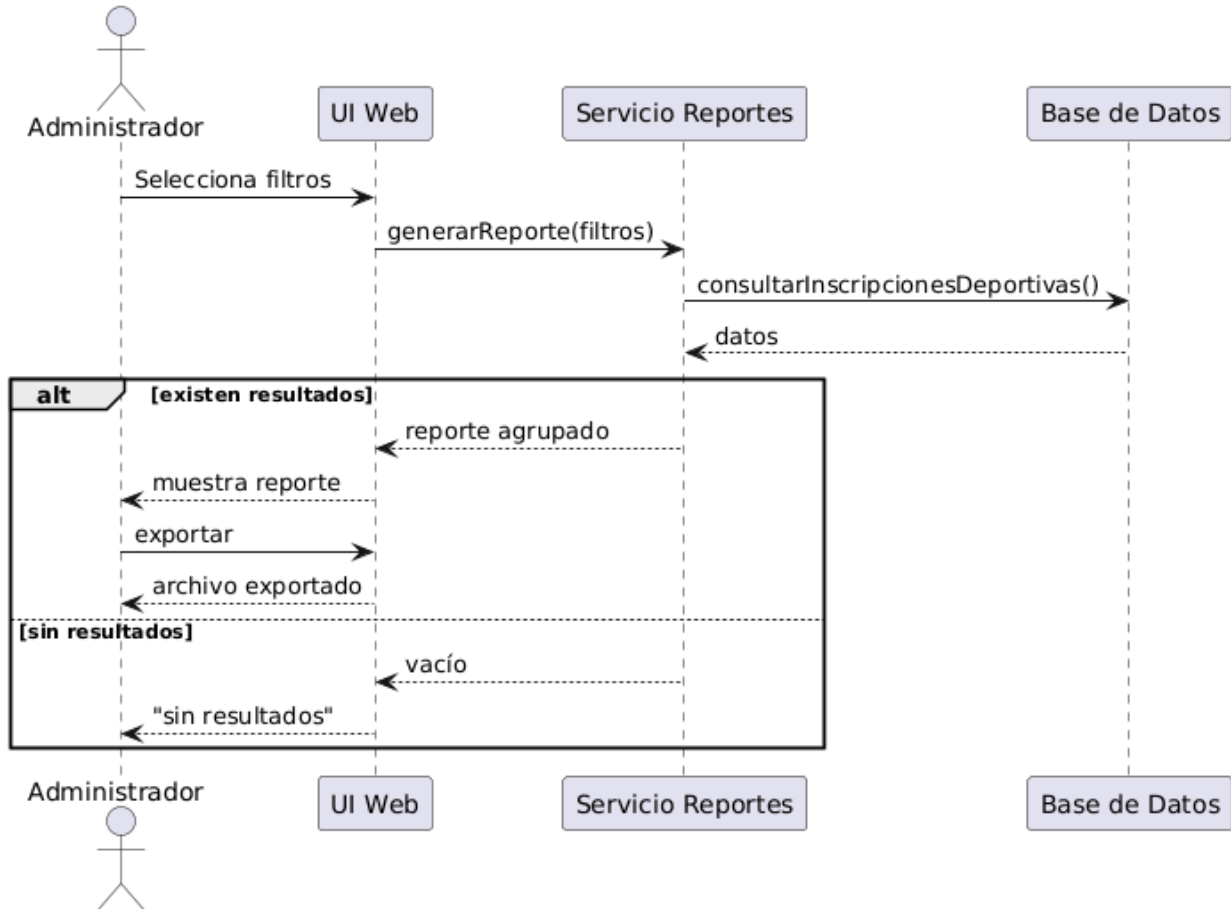
HISTORIA DE USUARIO — HU-06	
Número: HU-06	Usuario: Directivo/Administrador
Nombre historia: Generar reporte de alumnos por deporte, nivel, horario y profesor	
Prioridad en negocio: Media	Riesgo en desarrollo: Medio
Puntos estimados: 5	Iteración asignada: 3
Programador responsable: Integrante 2	
Descripción: Como directivo quiero un reporte de alumnos agrupado por deporte, nivel, horario y profesor, para controlar las actividades deportivas.	
Validación: El reporte agrupa por deporte-nivel-horario, muestra al profesor responsable y los alumnos inscriptos; puede exportarse.	

B) Caso de Uso

CASO DE USO — Generar Reporte de Alumnos por Deporte	
Autor: Bruno Mazza	Fecha: 11/08/2026
Prioridad: 3	Actores: Directivo/Administrador
Descripción: Permite obtener un listado consolidado de alumnos agrupados por deporte, nivel, horario y profesor.	
Precondiciones: Existen inscripciones a deportes registradas.	
Flujo normal: 1.- Accede a "Reportes" y selecciona este listado. 2.- Aplica filtros opcionales (deporte, nivel). 3.- El sistema consulta la información consolidada. 4.- Presenta el reporte agrupado en pantalla. 5.- Permite exportarlo (PDF/Excel).	
Flujo alternativo: 3.A.- Si no hay datos para los filtros, informa "sin resultados".	

Poscondiciones:

El reporte queda disponible en pantalla y, opcionalmente, exportado.

C) Diagrama de Secuencia (PlantUML)

@startuml

actor Administrador

participant "UI Web" as UI

participant "Servicio Reportes" as Svc

participant "Base de Datos" as DB

Administrador -> UI : Selecciona filtros

UI -> Svc : generarReporte(filtros)

Svc -> DB : consultarInscripcionesDeportivas()

DB --> Svc : datos

alt existen resultados

Svc --> UI : reporte agrupado

UI --> Administrador : muestra reporte

Administrador -> UI : exportar

```
UI --> Administrador : archivo exportado
else sin resultados
Svc --> UI : vacío
UI --> Administrador : "sin resultados"
end
@enduml
```

5. Desafío — Funcionalidad de Valor Agregado

Inicio de sesión en la app móvil desde la plataforma web

Se propone poder iniciar sesión desde la página a la app móvil y viceversa

- Ingresar en la web al apartado de inicio de sesión con QR
- En la app móvil seleccionar la ingresar con QR
- Escanear el QR de la web para autenticar al usuario e iniciar sesión

Credencial Digital con Código QR + Notificación Automática a Padres

Se propone una credencial digital (QR) por alumno, accesible desde el perfil web y la futura app, que permita:

- Registrar ingreso/egreso del alumno al establecimiento escaneando el QR en portería.
- Registrar la subida/bajada del micro escolar, vinculada al recorrido de transporte (RF-07).
- Enviar una notificación automática al padre por cada evento de ingreso/egreso.
- Alimentar un reporte de asistencia diaria por curso.

Reutiliza la arquitectura ya diseñada (permisos por rol, transporte, capa de servicios REST) y responde a una necesidad sensible: la seguridad y trazabilidad de los alumnos.